Simulating Brownian Motion, Geometric Brownian Motion, the Ornstein–Uhlenbeck Process and Short-Rate Models

Aaryan Pradhan (22CH3FP25)

MA30247 Financial Mathematics, Individual Project
Indian Institute of Technology Kharagpur

November 2026

Abstract

We simulate the processes set for this project: Brownian motion, geometric Brownian motion, the Ornstein–Uhlenbeck process and the short-rate (VIR) model. For the short rate we run two models, Vasicek and Cox–Ingersoll–Ross (CIR). Every process is drawn from its exact transition law, so the only error left is Monte Carlo noise. With 20,000 paths, all 19 simulated moments, probabilities and bond prices lie within 1.86 standard errors of their closed-form values. The plots show how each model behaves. The variance of Brownian motion grows linearly in time. Geometric Brownian motion is lognormal, and its median falls further below its mean as volatility rises. The Ornstein–Uhlenbeck process forgets its starting point with a half-life of $\ln 2/\theta$. The Vasicek rate goes below zero on 25.8% of paths within ten years, while a CIR rate with the same mean and long-run variance never does. Finally, we measure the strong error of the Euler and Milstein schemes for geometric Brownian motion and find orders 0.49 and 0.99, close to the theoretical 1/2 and 1.

Keywords: Brownian motion; Geometric Brownian motion; Ornstein–Uhlenbeck process; Vasicek model; CIR model; Exact simulation; Monte Carlo

1 Introduction

Most models in mathematical finance are stochastic differential equations (SDEs). A stock price, an interest rate or a spread is written as a drift term plus a noise term, and the noise comes from Brownian motion. A few of these models can be solved by hand. For the others, and for any payoff that depends on the whole path, we simulate.

This report simulates the processes listed in the project brief. Brownian motion is the noise source for all of them. Geometric Brownian motion (GBM) is the stock price model behind Samuelson [6] and Black and Scholes [1]. The Ornstein–Uhlenbeck (OU) process [8] adds a pull toward a long-run level. Vasicek [9] used the OU process as a model of the short interest rate. The brief calls the short-rate model "VIR". We read this as the Vasicek interest-rate model, and since the same abbreviation is sometimes used for the Cox–Ingersoll–Ross (CIR) model [2], we simulate both. The two differ in a single term, and that difference decides whether rates can turn negative.

Each of these models has a known solution, and we use it twice. First, we simulate from it: each time step is drawn from the exact conditional distribution, so there is no discretization error in the paths [3]. Second, we check against it: every simulated mean, variance, probability and bond price is compared with its formula, measured in units of its Monte Carlo standard error.

Section 2 states the models and their solutions. Section 3 describes the simulation. Section 4 goes through the results one process at a time, and Section 5 concludes. The code in the

appendix rebuilds every figure and number in this report in a few seconds. Figure 1 shows one path of each process.

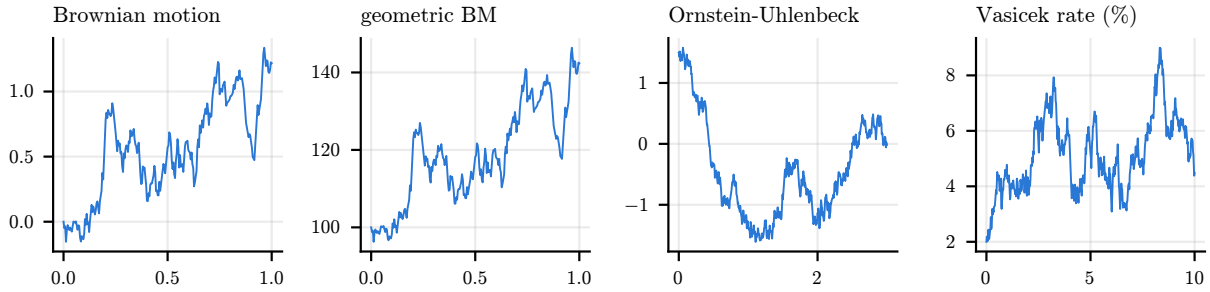


Figure 1: One simulated path of each process. Parameters are in Table 2. Brownian motion and GBM wander off with no level to return to; the OU process and the Vasicek rate are pulled back toward their long-run levels (0 and 5%).

2 The models

Throughout, $(W_t)_{t\geq 0}$ is a standard Brownian motion and Z is a standard normal random variable. Table 1 at the end of the section collects the results.

2.1 Brownian motion

A standard Brownian motion has $W_0 = 0$, continuous paths, and independent increments with

$$W_t - W_s \sim N(0,\, t-s), \qquad 0 \leq s < t. \tag{1}$$

So $\mathbb{E}[W_t] = 0$, $\mathrm{Var}[W_t] = t$ and $\mathrm{Cov}(W_s, W_t) = \min(s,t)$. The spread of W_t grows like $\sqrt{t}$, not like t.

Two path properties matter for what follows. Take a partition $0 = t_0 < t_1 < \cdots < t_n = T$ whose largest step goes to zero. Then the *quadratic variation* $\sum_k (W_{t_{k+1}} - W_{t_k})^2$ converges to T, while the total variation $\sum_k |W_{t_{k+1}} - W_{t_k}|$ goes to infinity. A Brownian path is too rough for ordinary calculus. The finite quadratic variation is the source of the extra $\frac{1}{2}f''$ term in Itô's formula [7].

2.2 Geometric Brownian motion

A stock price with expected return μ and volatility σ follows

$$dS_t = \mu S_t\, dt + \sigma S_t\, dW_t, \qquad S_0 > 0. \tag{2}$$

Itô's formula applied to $\ln S_t$ gives $d\ln S_t = (\mu - \frac{1}{2}\sigma^2)\, dt + \sigma\, dW_t$. The right side has constant coefficients, so we can integrate it directly:

$$S_t = S_0 \exp\Big(\big(\mu - \tfrac{1}{2}\sigma^2\big)t + \sigma W_t \Big). \tag{3}$$

Hence $\ln S_t$ is normal and S_t is lognormal. It is always positive, and

$$\mathbb{E}[S_t] = S_0 e^{\mu t}, \qquad \mathrm{median}(S_t) = S_0 e^{(\mu - \sigma^2/2)t}, \qquad \mathrm{Var}[S_t] = S_0^2 e^{2\mu t}\big(e^{\sigma^2 t} - 1\big). \tag{4}$$

The mean is larger than the median by the factor $e^{\sigma^2 t/2}$. A few very good paths pull the mean up, while most paths end below it.

2.3 The Ornstein–Uhlenbeck process

The OU process is pulled toward a level μ at speed $\theta > 0$:

$$dX_t = \theta(\mu - X_t)\,dt + \sigma\,dW_t. \tag{5}$$

Multiply by $e^{\theta t}$. Itô's product rule gives $d(e^{\theta t}X_t) = \theta\mu e^{\theta t}\,dt + \sigma e^{\theta t}\,dW_t$, and integrating from 0 to t,

$$X_t = \mu + (X_0 - \mu)e^{-\theta t} + \sigma\int_0^t e^{-\theta(t-s)}\,dW_s. \tag{6}$$

The stochastic integral has a deterministic integrand, so it is normal with mean zero. By the Itô isometry its variance is $\sigma^2\int_0^t e^{-2\theta(t-s)}ds$. Therefore

$$\mathbb{E}[X_t] = \mu + (X_0 - \mu)e^{-\theta t}, \qquad \mathrm{Var}[X_t] = \frac{\sigma^2}{2\theta}\big(1 - e^{-2\theta t}\big). \tag{7}$$

As $t \to \infty$ the law of X_t tends to $N(\mu, \sigma^2/2\theta)$, the *stationary* distribution. The expected distance to μ halves every $\ln 2/\theta$ units of time. If X_0 is drawn from the stationary law, then $\mathrm{corr}(X_s, X_{s+\tau}) = e^{-\theta\tau}$.

2.4 Short-rate models (VIR): Vasicek and CIR

Let r_t be the instantaneous short interest rate. Vasicek [9] takes

$$dr_t = a(b - r_t)\,dt + \sigma\,dW_t, \tag{8}$$

which is (5) with $\theta = a$ and $\mu = b$. So r_t is normal with mean $b + (r_0 - b)e^{-at}$ and variance $\frac{\sigma^2}{2a}(1 - e^{-2at})$. A normal variable can be negative, so $\mathbb{P}(r_t < 0) = \Phi\big(-\mathbb{E}[r_t]/\sqrt{\mathrm{Var}[r_t]}\big) > 0$ for every $t > 0$.

The price at time 0 of a zero-coupon bond paying 1 at time T is $P(0,T) = \mathbb{E}\big[\exp(-\int_0^T r_s\,ds)\big]$, where we treat (8) as the risk-neutral dynamics. For Vasicek this expectation has the closed form

$$P(0,T) = A(T)e^{-B(T)r_0}, \quad B(T) = \frac{1 - e^{-aT}}{a}, \quad \ln A(T) = \Big(b - \frac{\sigma^2}{2a^2}\Big)\big(B(T) - T\big) - \frac{\sigma^2 B(T)^2}{4a}. \tag{9}$$

The yield $y(0,T) = -\ln P(0,T)/T$ tends to $b - \sigma^2/(2a^2)$ as $T \to \infty$.

The CIR model [2] multiplies the noise by $\sqrt{r_t}$:

$$dr_t = a(b - r_t)\,dt + \sigma\sqrt{r_t}\,dW_t. \tag{10}$$

As r_t approaches zero the noise dies out and the drift $ab > 0$ pushes the rate back up. If the Feller condition $2ab \geq \sigma^2$ holds, the rate never reaches zero. The mean is the same as in Vasicek. The variance is $r_0\frac{\sigma^2}{a}(e^{-at} - e^{-2at}) + \frac{b\sigma^2}{2a}(1 - e^{-at})^2$, and r_t is a scaled noncentral chi-square variable (Section 3). The CIR bond price also has the form $A(T)e^{-B(T)r_0}$, now with $h = \sqrt{a^2 + 2\sigma^2}$,

$$B(T) = \frac{2(e^{hT} - 1)}{(h + a)(e^{hT} - 1) + 2h}, \qquad A(T) = \left(\frac{2h\,e^{(a+h)T/2}}{(h + a)(e^{hT} - 1) + 2h}\right)^{2ab/\sigma^2}. \tag{11}$$

Table 1: The models at a glance.

Model	SDE	Law of X_t	$\mathbb{E}[X_t]$	Values
BM	dW_t	normal	0	$\mathbb{R}$
GBM	$\mu S\,dt + \sigma S\,dW$	lognormal	$S_0 e^{\mu t}$	$(0,\infty)$
OU	$\theta(\mu - X)\,dt + \sigma\,dW$	normal	$\mu + (X_0-\mu)e^{-\theta t}$	$\mathbb{R}$
Vasicek	$a(b-r)\,dt + \sigma\,dW$	normal	$b + (r_0-b)e^{-at}$	$\mathbb{R}$
CIR	$a(b-r)\,dt + \sigma\sqrt{r}\,dW$	scaled χ'^2	$b + (r_0-b)e^{-at}$	$(0,\infty)$ if $2ab \ge \sigma^2$

3 Simulation method

We use an equally spaced grid $t_k = k\,\Delta t$ and independent draws $Z_k \sim N(0,1)$. Each update below is the exact conditional law of the next value given the current one, so the simulated values on the grid have exactly the right joint distribution [3]:

$$\text{BM:}\quad W_{k+1} = W_k + \sqrt{\Delta t}\,Z_k, \tag{12}$$

$$\text{GBM:}\quad S_{k+1} = S_k \exp\big((\mu - \tfrac{1}{2}\sigma^2)\Delta t + \sigma\sqrt{\Delta t}\,Z_k\big), \tag{13}$$

$$\text{OU, Vasicek:}\quad X_{k+1} = \mu + (X_k-\mu)e^{-\theta\Delta t} + \sigma\sqrt{\tfrac{1-e^{-2\theta\Delta t}}{2\theta}}\,Z_k, \tag{14}$$

$$\text{CIR:}\quad r_{k+1} = c\,\chi'^2_\nu(\lambda_k),\quad c = \tfrac{\sigma^2(1-e^{-a\Delta t})}{4a},\ \nu = \tfrac{4ab}{\sigma^2},\ \lambda_k = \tfrac{r_k e^{-a\Delta t}}{c}. \tag{15}$$

Here $\chi'^2_\nu(\lambda)$ is a noncentral chi-square draw with ν degrees of freedom and noncentrality λ. GBM is computed from (3) on the same Brownian paths, which is the same thing as the GBM update above. For bond prices we need $\int_0^T r_s\,ds$ on each path, which we approximate with the trapezoid rule. This is the only discretization error in the study, and Table 3 shows it is smaller than the Monte Carlo noise.

Euler and Milstein. To see what exact simulation saves, we also run the two standard schemes on GBM [5, 4], with $\Delta W_k = \sqrt{\Delta t}\,Z_k$:

$$\text{Euler–Maruyama:}\quad S_{k+1} = S_k + \mu S_k\Delta t + \sigma S_k \Delta W_k, \tag{16}$$

$$\text{Milstein:}\quad S_{k+1} = S_k + \mu S_k\Delta t + \sigma S_k \Delta W_k + \tfrac{1}{2}\sigma^2 S_k\big(\Delta W_k^2 - \Delta t\big). \tag{17}$$

We drive both with the same Brownian increments as the exact solution and measure the strong error $\mathbb{E}\big|S_T^{\Delta t} - S_T\big|$. Theory says it shrinks like $\Delta t^{1/2}$ for Euler and like Δt for Milstein.

Checking against theory. For a quantity $\mathbb{E}[f]$ we report the sample mean $\hat{m}$ of f over M paths, its standard error $\hat{s}/\sqrt{M}$, and $z = (\hat{m} - \text{theory})/(\hat{s}/\sqrt{M})$. If the code is correct, z is roughly standard normal. A variance is checked as the mean of $(X - \mathbb{E}X)^2$, a probability as the mean of an indicator and a bond price as the mean of $e^{-\int r}$. The code stops if any $|z|$ exceeds 4.

Table 2: Parameters. All runs use $M = 20{,}000$ paths and the PCG64 generator with seed 2026.

Process	Parameters	Horizon	Step
BM	$W_0 = 0$	$T = 1$	$\Delta t = 1/252$ (daily)
GBM	$S_0 = 100$, $\mu = 0.08$, $\sigma = 0.25$	$T = 1$	$1/252$
OU	$X_0 = 1.5$, $\theta = 2$, $\mu = 0$, $\sigma = 1$	$T = 3$	$1/200$
Vasicek	$r_0 = 2\%$, $a = 0.5$, $b = 5\%$, $\sigma = 0.02$	$T = 10$	$1/52$ (weekly)
CIR	$r_0 = 2\%$, $a = 0.5$, $b = 5\%$, $\sigma = 0.0894$	$T = 10$	$1/52$

The CIR volatility is set to $\sigma_{\text{CIR}} = \sigma_{\text{Vas}}/\sqrt{b}$. With this choice both rate models have the same mean at every date and the same long-run variance, $\sigma^2_{\text{Vas}}/2a$, so any difference between them comes from the shape of the noise. The Feller condition holds: $2ab = 0.050 \geq \sigma^2_{\text{CIR}} = 0.0080$.

4 Results

4.1 Brownian motion

Figure 2 shows ten paths with the 50% and 90% bands of all 20,000. The bands open like $\sqrt{t}$. The outer band lies just inside the $\pm 2\sqrt{t}$ curves, at $\pm 1.645\sqrt{t}$, because W_t is $N(0, t)$ and 90% of a normal law lies within 1.645 standard deviations.

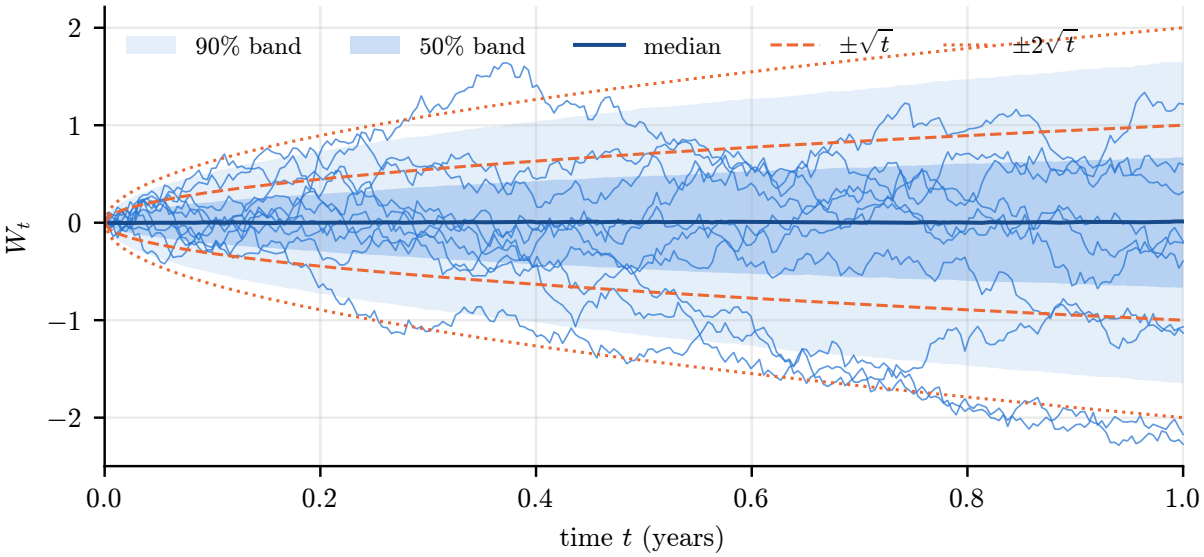


Figure 2: Brownian motion. Ten sample paths (thin), the 50% and 90% bands across all paths, and the curves $\pm\sqrt{t}$ and $\pm 2\sqrt{t}$.

Figure 3 tests the definition. Panel (a) matches the histogram of W_1 to the $N(0, 1)$ density. Panel (b) shows the sample variance across paths lying on the line t. Panel (c) takes one path on 2^{16} steps and computes both variations on coarser and finer grids. The sum of squared increments settles at $T = 1$ (0.997 on the finest grid). The sum of absolute increments keeps growing like $\sqrt{n}$ and reaches 204. This is the roughness described in Section 2, seen on an actual path.

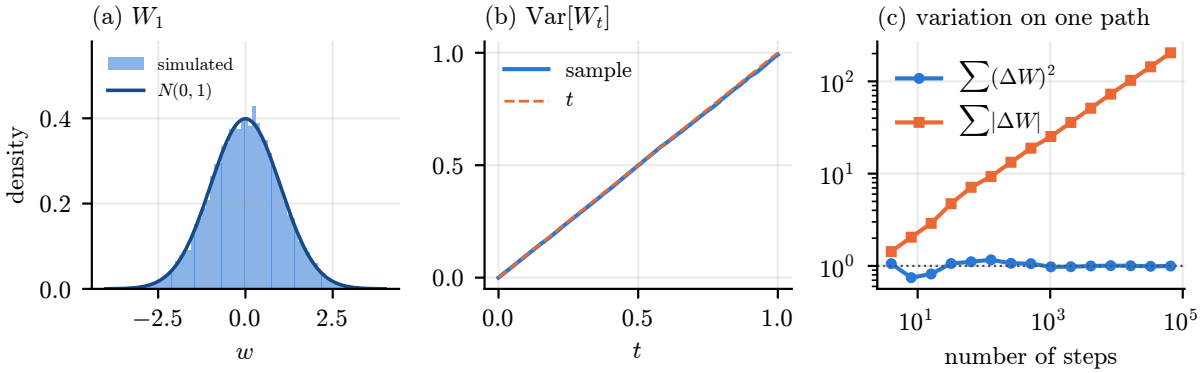


Figure 3: (a) Distribution of W_1. (b) Var$[W_t]$ across paths against t. (c) Quadratic variation (blue) and total variation (orange) of one path as the grid is refined.

4.2 Geometric Brownian motion

Figure 4 uses the same Brownian paths as Figure 2. The paths stay positive and the bands are no longer symmetric: the upper band is wider than the lower one. The terminal histogram in panel (b) is skewed to the right and lies on the lognormal density. The mean of S_1 is $S_0 e^{\mu} = 108.33$, above the median 105.00. Across all paths S_1 ranges from 39.8 to 273.6.

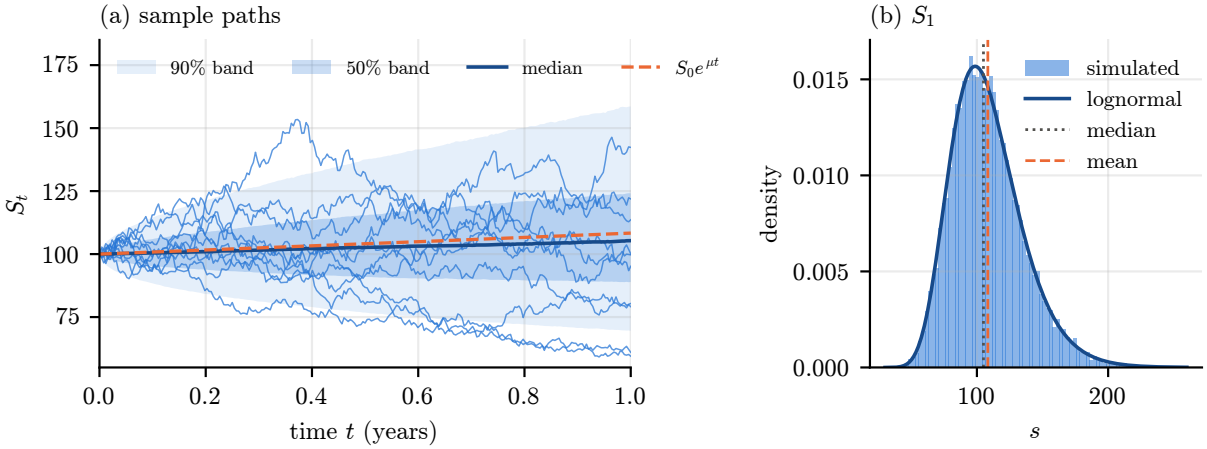


Figure 4: Geometric Brownian motion with $S_0 = 100$, $\mu = 0.08$, $\sigma = 0.25$. (a) Paths, bands, median and the mean curve $S_0 e^{\mu t}$. (b) Distribution of S_1 against the lognormal density, with its mean and median.

Figure 5 changes only σ. In panel (a) one set of shocks drives three prices: a larger σ gives larger swings in both directions. Panel (b) shows the effect on the distribution of S_1. All three densities have the same mean, $S_0 e^{\mu}$, because σ does not appear in $\mathbb{E}[S_t]$. The median, marked by a triangle, moves left as σ grows. The typical outcome gets worse while the average stays put, and the gap is paid for by a long right tail. This is why a volatile asset with a high expected return can still lose money on most paths.

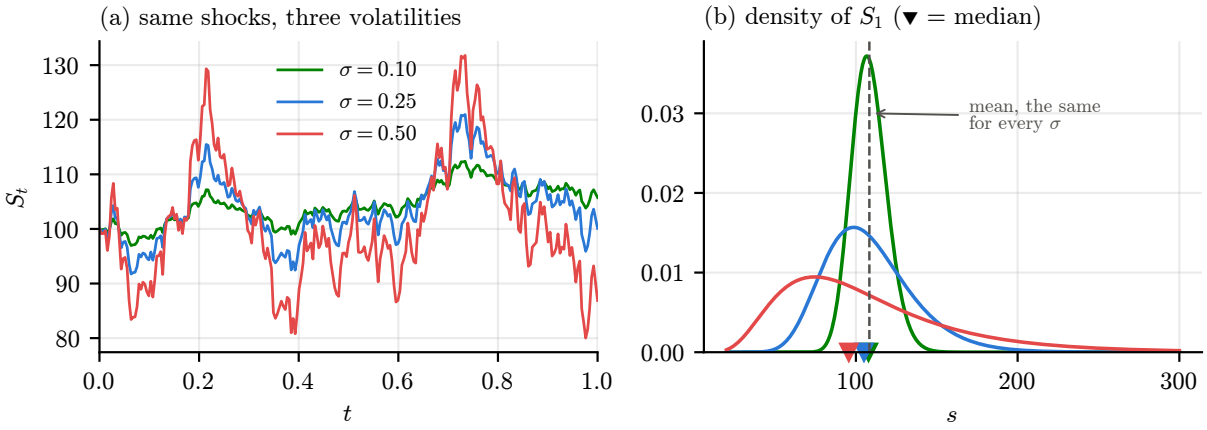


Figure 5: Effect of volatility in GBM. (a) The same Brownian path with $\sigma = 0.10$, 0.25 and 0.50. (b) Lognormal densities of S_1; triangles mark the medians, the dashed line the common mean.

Figure 6 compares the two time-stepping schemes with the exact solution. On a log-log scale the errors are straight lines. The fitted slopes are 0.49 for Euler and 0.99 for Milstein, in line with the strong orders 1/2 and 1. Halving the step cuts the Euler error by only about 30%. The Milstein error halves.

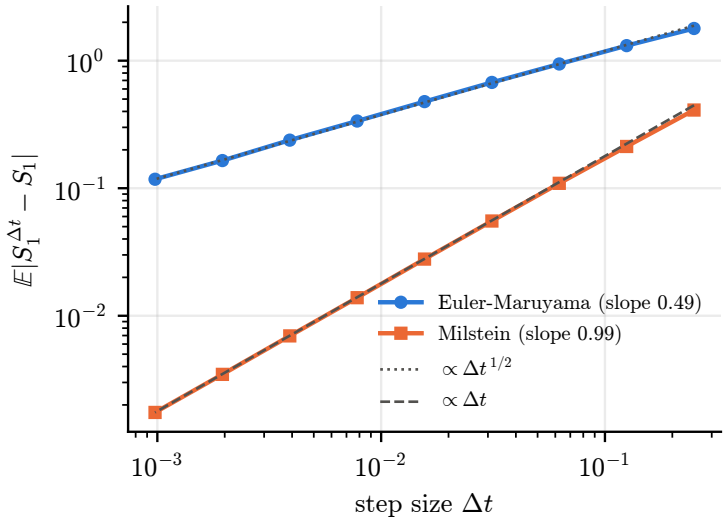


Figure 6: Strong error at $T = 1$ of the Euler–Maruyama and Milstein schemes for GBM, against the exact solution driven by the same shocks (5,000 paths). Grey lines show slopes 1/2 and 1.

4.3 The Ornstein–Uhlenbeck process

Figure 7(a) starts the OU process from -1.5, 0 and 1.5. Paths from all three starting points end up in the same band around $\mu = 0$, and the thick curves are the theoretical means $\mu + (X_0 - \mu)e^{-\theta t}$. With $\theta = 2$ the half-life is $\ln 2/\theta = 0.35$. After about 1.5 time units, four half-lives, less than 7% of the starting gap is left. Panel (b) shows the whole distribution of X_t over time as a heat map, built from all 20,000 paths started at 1.5. It is narrow at the start, slides down toward 0, and stops widening once it reaches the stationary spread. The dashed lines are the mean ± 2 standard deviations from (7), and they follow the edge of the dark region.

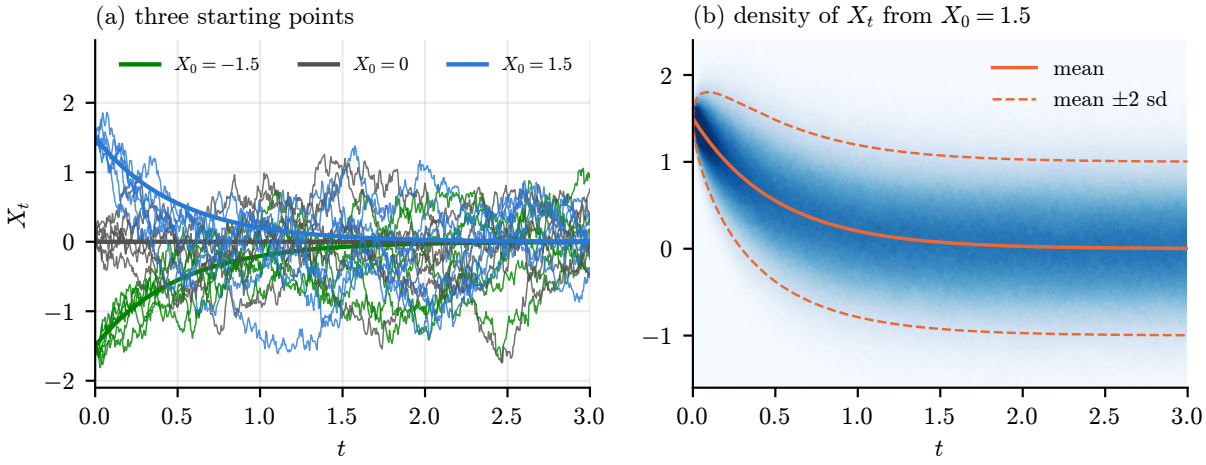


Figure 7: OU process with $\theta = 2$, $\mu = 0$, $\sigma = 1$. (a) Six paths from each of three starting points, with the theoretical means. (b) Density of X_t over time from $X_0 = 1.5$ (darker means more paths), with the mean and the mean ± 2 standard deviations.

Figure 8 looks at the long run. In panel (a) the distribution of X_3 matches the stationary law $N(0, \sigma^2/2\theta) = N(0, 0.25)$. In panel (b) we start paths from that law and measure the correlation between X_0 and X_τ. It decays as $e^{-\theta\tau}$, so the process keeps a short memory. Panel (c) feeds the same shocks into three speeds of mean reversion. A larger θ pulls the path to 0 faster and keeps it in a narrower range, since the stationary standard deviation is $\sigma/\sqrt{2\theta}$.

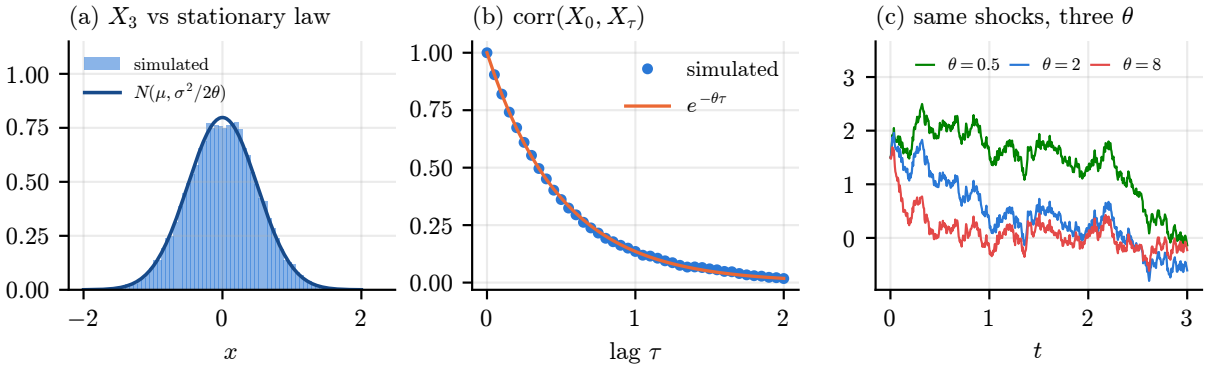


Figure 8: Long-run behaviour of the OU process. (a) X_3 against the stationary density. (b) Autocorrelation under the stationary law. (c) One shock path with $\theta = 0.5$, 2 and 8.

4.4 Short-rate models (VIR): Vasicek and CIR

Figure 9 shows ten years of weekly short rates. Both models start at 2% and rise toward 5% along the same mean curve. They spread differently. In Vasicek the noise has the same size at every level, so the band is symmetric and part of it lies below zero. In CIR the noise is proportional to $\sqrt{r_t}$. It is small when rates are low and large when they are high, which gives a band squeezed at the bottom and stretched at the top. The lowest Vasicek value across all paths and weeks is -5.26%. The lowest CIR value is 0.159%.

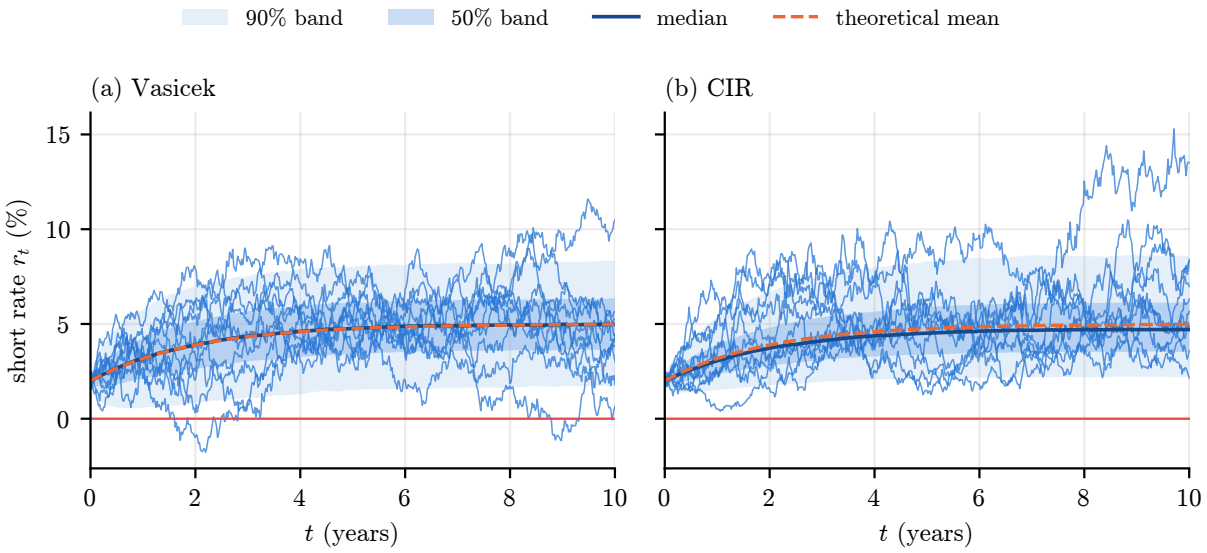


Figure 9: Short rate over ten years under (a) Vasicek and (b) CIR, with the same mean and the same long-run variance. The red line is zero.

Figure 10(a) puts the two ten-year distributions on top of each other. They have the same mean and nearly the same variance, but Vasicek is a symmetric normal with a left tail below zero, while CIR is skewed to the right and stops at zero. Panel (b) tracks $\mathbb{P}(r_t < 0)$. For Vasicek it climbs above 2% in the first two years, while the rate is still near its low starting point and the variance is growing. It then settles near its long-run value $\Phi(-b/\sqrt{\sigma^2/2a})$. At ten years the formula gives 0.64% and the simulation 0.57%. Looked at path by path, 25.8% of Vasicek paths dip below zero at least once in the ten years. No CIR path does.

Panel (c) plots yield curves from the bond formulas (9) and (11) for three starting rates, with the Monte Carlo prices from our paths as markers. Starting below b gives a rising curve, and starting above b gives a falling one. At $r_0 = b$ the curve bends slightly down toward its

long-run limit $b - \sigma^2/2a^2 = 4.92\%$, because uncertainty about future rates raises bond prices and lowers yields. The simulated yields lie on the formula, and the Vasicek and CIR curves almost coincide. For pricing bonds the two models agree. Where they differ is the left tail of the rate itself.

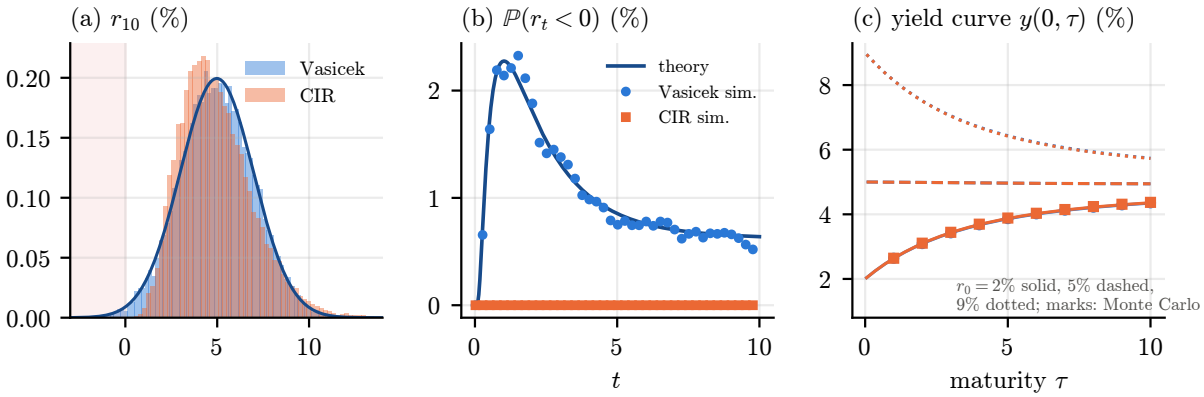


Figure 10: Vasicek (blue) and CIR (orange). (a) Distribution of r_{10}; the line is the Vasicek normal density and the shaded area is below zero. (b) Probability of a negative rate over time. (c) Yield curves from the closed forms for $r_0 = 2\%$, 5%, 9%; markers are Monte Carlo yields for $r_0 = 2\%$.

4.5 Accuracy

Table 3 lists every check. Each simulated value is within 1.86 standard errors of its formula. With 19 checks, a largest $|z|$ of about 2 is what pure chance would produce, so the table shows no sign of bias. The bond prices are included even though they carry the trapezoid error from Section 3, and they pass too.

Table 3: Simulated values against closed-form values. SE is the Monte Carlo standard error and $z = (\text{simulated} - \text{theory})/\text{SE}$.

Model	Quantity	Theory	Simulated	SE	z
BM	$\mathbb{E}[W_1]$	0	0.0041667	0.007	+0.59
	$\mathrm{Var}[W_1]$	1	0.99135	0.0099	−0.87
	$\mathbb{E}[W_{1/2} W_1] = \min(s, t)$	0.5	0.49382	0.0061	−1.02
GBM	$\mathbb{E}[S_1]$	108.33	108.41	0.19	+0.40
	$\mathrm{Var}[S_1]$	756.85	746.77	9	−1.12
	$\mathbb{P}(S_1 \le \text{median})$	0.5	0.494	0.0035	−1.70
	$\mathbb{E}[\ln S_1]$	4.6539	4.655	0.0018	+0.59
OU	$\mathbb{E}[X_1]$	0.203	0.20342	0.0035	+0.12
	$\mathrm{Var}[X_1]$	0.24542	0.24465	0.0024	−0.32
	$\mathrm{Var}[X_3]$ (near $\sigma^2/2\theta$)	0.25	0.24976	0.0025	−0.10
Vasicek	$\mathbb{E}[r_1]$	0.031804	0.031883	0.00011	+0.70
	$\mathrm{Var}[r_1]$	0.00025285	0.00024979	2.5×10^{-6}	−1.24
	$P(0, 5)$	0.82443	0.82419	0.00036	−0.65
	$P(0, 10)$	0.64741	0.64694	0.00049	−0.95
	$\mathbb{P}(r_{10} < 0)$	0.0063881	0.00575	0.00053	−1.19
CIR	$\mathbb{E}[r_1]$	0.031804	0.031875	8.3×10^{-5}	+0.85
	$\mathrm{Var}[r_1]$	0.0001383	0.00013876	1.7×10^{-6}	+0.28
	$P(0, 5)$	0.82395	0.82347	0.00029	−1.65
	$P(0, 10)$	0.64676	0.64596	0.00043	−1.86

5 Conclusion

We simulated Brownian motion, geometric Brownian motion, the Ornstein–Uhlenbeck process and the Vasicek and CIR short-rate models from their exact transition laws, and every simulated quantity agrees with its formula within Monte Carlo error. The simulations make the differences between the models easy to see. Brownian motion and GBM have no level to return to, and their spread grows without bound. GBM stays positive, but its median drifts below its mean as volatility rises. The OU process and the two rate models settle into a stationary band, and the OU process loses track of where it started within a few half-lives. Among the rate models, Vasicek and CIR give almost the same bond prices, but only CIR keeps the rate positive. Vasicek allows negative rates on about a quarter of the paths over ten years.

Exact simulation was possible here because each model has a closed-form transition law. For models without one, a scheme such as Euler or Milstein is needed. Our GBM test shows the cost: Euler converges at order $1/2$, so it needs about four times as many steps to halve its error.

References

[1] Fischer Black and Myron Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654, 1973.

[2] John C. Cox, Jonathan E. Ingersoll, and Stephen A. Ross. A theory of the term structure of interest rates. *Econometrica*, 53(2):385–407, 1985.

[3] Paul Glasserman. *Monte Carlo Methods in Financial Engineering*. Springer, New York, 2003.

[4] Desmond J. Higham. An algorithmic introduction to numerical simulation of stochastic differential equations. *SIAM Review*, 43(3):525–546, 2001.

[5] Peter E. Kloeden and Eckhard Platen. *Numerical Solution of Stochastic Differential Equations*. Springer, Berlin, 1992.

[6] Paul A. Samuelson. Rational theory of warrant pricing. *Industrial Management Review*, 6(2):13–39, 1965.

[7] Steven E. Shreve. *Stochastic Calculus for Finance II: Continuous-Time Models*. Springer, New York, 2004.

[8] George E. Uhlenbeck and Leonard S. Ornstein. On the theory of the Brownian motion. *Physical Review*, 36(5):823–841, 1930.

[9] Oldrich Vasicek. An equilibrium characterization of the term structure. *Journal of Financial Economics*, 5(2):177–188, 1977.

A Code

The script below produces every figure and every number in this report (`uv run -with numpy -with matplotlib python sim.py`).

```
"""Monte Carlo simulation of BM, GBM, OU, Vasicek and CIR.
Writes every figure (figures/) and every number (tables/) that main.tex uses.

    uv run --no-project --with numpy --with matplotlib python sim.py
"""
import os
```

```
from math import erf, sqrt
import numpy as np
import matplotlib as mpl
import matplotlib.pyplot as plt
from matplotlib.colors import PowerNorm

HERE = os.path.dirname(os.path.abspath(__file__))
FIG, TAB = os.path.join(HERE, "figures"), os.path.join(HERE, "tables")
os.makedirs(FIG, exist_ok=True)
os.makedirs(TAB, exist_ok=True)

BLUE, DARK, ORANGE, GREY, GREEN, RED = "#2A78D6", "#174A8A", "#EB6834", "#52514E", "#008300", "#E34948"
mpl.rcParams.update({
    "font.family": "serif", "font.serif": ["cmr10"], "axes.formatter.use_mathtext": True,
    "mathtext.fontset": "cm", "axes.unicode_minus": False, "pdf.fonttype": 42, "font.size": 9, "axes.
        titlesize": 9,
    "axes.titlelocation": "left", "axes.spines.top": False, "axes.spines.right": False,
    "axes.grid": True, "grid.alpha": 0.25, "legend.frameon": False, "legend.fontsize": 8,
    "savefig.bbox": "tight", "savefig.pad_inches": 0.02,
})

rng = np.random.default_rng(2026)
M = 20_000 # Monte Carlo paths
Phi = np.vectorize(lambda x: 0.5 * (1 + erf(x / sqrt(2))))
npdf = lambda x, m, s: np.exp(-(x - m) ** 2 / (2 * s * s)) / (s * sqrt(2 * np.pi))
rows, num = [], {}


def save(fig, name):
    fig.savefig(os.path.join(FIG, name + ".pdf"))
    plt.close(fig)


def check(group, name, f, theory):
    """Monte Carlo estimate of E[f] with standard error. Fails if it is more than 4 SE from theory."""
    est, se = f.mean(), f.std(ddof=1) / sqrt(len(f))
    z = (est - theory) / se
    assert abs(z) < 4, (name, est, theory, se)
    rows.append((group, name, theory, est, se, z))


def fan(ax, t, X, color, k=10):
    q = np.percentile(X, [5, 25, 50, 75, 95], axis=0)
    ax.fill_between(t, q[0], q[4], color=color, alpha=0.12, lw=0, label="90% band")
    ax.fill_between(t, q[1], q[3], color=color, alpha=0.25, lw=0, label="50% band")
    ax.plot(t, X[:k].T, color=color, lw=0.55, alpha=0.75)
    ax.plot(t, q[2], color=DARK, lw=1.3, label="median")


def paths(x0, step, Z):
    X = np.empty((Z.shape[0], Z.shape[1] + 1))
    X[:, 0] = x0
    for i in range(Z.shape[1]):
        X[:, i + 1] = step(X[:, i], Z[:, i])
    return X


def ou_step(th, m, s, dt):
    """Exact OU transition: X(t+dt) | X(t) is normal."""
    e, sd = np.exp(-th * dt), s * sqrt((1 - np.exp(-2 * th * dt)) / (2 * th))
    return lambda x, z: m + (x - m) * e + sd * z


# ------------------------------------------------------------- 1. Brownian motion
T, n = 1.0, 252
dt = T / n
t = np.linspace(0, T, n + 1)
Z = rng.standard_normal((M, n))
W = np.concatenate([np.zeros((M, 1)), sqrt(dt) * Z.cumsum(1)], 1)
check("BM", r"$\mathbb{E}[W_1]$", W[:, -1], 0.0)
check("BM", r"$\mathrm{Var}[W_1]$", W[:, -1] ** 2, T)
check("BM", r"$\mathbb{E}[W_{1/2}W_1]=\min(s,t)$", W[:, n // 2] * W[:, -1], 0.5)

```

```
fig, ax = plt.subplots(figsize=(6.3, 2.7))
fan(ax, t, W, BLUE)
for c, ls in [(1, "--"), (2, ":")]:
    ax.plot(t, c * np.sqrt(t), color=ORANGE, ls=ls, lw=1, label=rf"$\pm{c}\sqrt{{t}}$" if c > 1 else r"$\pm \sqrt{t}$")
    ax.plot(t, -c * np.sqrt(t), color=ORANGE, ls=ls, lw=1)
ax.set(xlabel="time␣$t$␣(years)", ylabel="$W_t$", xlim=(0, T))
ax.legend(ncol=5, loc="upper␣left")
save(fig, "bm_paths")

fine = 2 ** 16
dWf = rng.standard_normal(fine) * sqrt(T / fine)
Wf = np.concatenate([[0], dWf.cumsum()])
ks = np.arange(2, 17)
qv = [np.sum(np.diff(Wf[:: fine // 2 ** k]) ** 2) for k in ks]
tv = [np.sum(np.abs(np.diff(Wf[:: fine // 2 ** k]))) for k in ks]
num["QVFinest"] = f"{qv[-1]:.3f}"
num["TVFinest"] = f"{tv[-1]:.0f}"

fig, axs = plt.subplots(1, 3, figsize=(6.3, 2.1))
x = np.linspace(-4, 4, 300)
axs[0].hist(W[:, -1], bins=70, density=True, color=BLUE, alpha=0.55, label="simulated")
axs[0].plot(x, npdf(x, 0, 1), color=DARK, lw=1.3, label="$N(0,1)$")
axs[0].set(title="(a)␣$W_1$", xlabel="$w$", ylabel="density", ylim=(0, 0.58))
axs[0].legend(loc="upper␣left", fontsize=6.5)
axs[1].plot(t, W.var(0), color=BLUE, lw=1.5, label="sample")
axs[1].plot(t, t, color=ORANGE, ls="--", lw=1, label="$t$")
axs[1].set(title=r"(b)␣$\mathrm{Var}[W_t]$", xlabel="$t$")
axs[1].legend()
axs[2].loglog(2.0 ** ks, qv, "o-", color=BLUE, ms=3, label=r"$\sum(\Delta␣W)^2$")
axs[2].loglog(2.0 ** ks, tv, "s-", color=ORANGE, ms=3, label=r"$\sum|\Delta␣W|$")
axs[2].axhline(T, color=GREY, lw=0.8, ls=":")
axs[2].set(title="(c)␣variation␣on␣one␣path", xlabel="number␣of␣steps")
axs[2].legend()
fig.tight_layout()
save(fig, "bm_checks")

# ------------------------------------------------------------- 2. Geometric Brownian motion
S0, mu, sig = 100.0, 0.08, 0.25
S = S0 * np.exp((mu - sig ** 2 / 2) * t + sig * W) # exact solution on the same W
mean_th, med_th = S0 * np.exp(mu * T), S0 * np.exp((mu - sig ** 2 / 2) * T)
check("GBM", r"$\mathbb{E}[S_1]$", S[:, -1], mean_th)
check("GBM", r"$\mathrm{Var}[S_1]$", (S[:, -1] - mean_th) ** 2, S0 ** 2 * np.exp(2 * mu * T) * (np.exp(sig ** 2 * T) - 1))
check("GBM", r"$\mathbb{P}(S_1\le␣\mathrm{median})$", (S[:, -1] <= med_th).astype(float), 0.5)
check("GBM", r"$\mathbb{E}[\ln␣S_1]$", np.log(S[:, -1]), np.log(S0) + (mu - sig ** 2 / 2) * T)
num["GBMMeanTh"], num["GBMMedTh"] = f"{mean_th:.2f}", f"{med_th:.2f}"
num["GBMMin"], num["GBMMax"] = f"{S[:,␣-1].min():.1f}", f"{S[:,␣-1].max():.1f}"

lnpdf = lambda s, m, v: np.exp(-(np.log(s) - m) ** 2 / (2 * v)) / (s * np.sqrt(2 * np.pi * v))
fig, axs = plt.subplots(1, 2, figsize=(6.3, 2.5), gridspec_kw={"width_ratios": [1.6, 1]})
fan(axs[0], t, S, BLUE)
axs[0].plot(t, S0 * np.exp(mu * t), color=ORANGE, ls="--", lw=1.2, label=r"$S_0e^{\mu␣t}$")
axs[0].set(title="(a)␣sample␣paths", xlabel="time␣$t$␣(years)", ylabel="$S_t$", xlim=(0, T), ylim=(55, 185))
axs[0].legend(ncol=4, loc="upper␣left", fontsize=7)
s = np.linspace(30, 260, 400)
axs[1].hist(S[:, -1], bins=90, range=(30, 260), density=True, color=BLUE, alpha=0.55, label="simulated")
axs[1].plot(s, lnpdf(s, np.log(S0) + (mu - sig ** 2 / 2) * T, sig ** 2 * T), color=DARK, lw=1.3, label="lognormal")
axs[1].axvline(med_th, color=GREY, ls=":", lw=1, label="median")
axs[1].axvline(mean_th, color=ORANGE, ls="--", lw=1, label="mean")
axs[1].set(title="(b)␣$S_1$", xlabel="$s$", ylabel="density")
axs[1].legend()
fig.tight_layout()
save(fig, "gbm_paths")

fig, axs = plt.subplots(1, 2, figsize=(6.3, 2.4))
for sg, c in zip([0.10, 0.25, 0.50], [GREEN, BLUE, RED]):
    axs[0].plot(t, S0 * np.exp((mu - sg ** 2 / 2) * t + sg * W[1]), color=c, lw=1, label=rf"$\sigma={sg:.2f}$")
    s = np.linspace(20, 300, 500)
    axs[1].plot(s, lnpdf(s, np.log(S0) + (mu - sg ** 2 / 2) * T, sg ** 2 * T), color=c, lw=1.3, label=rf"$
```

```
          \sigma={sg:.2f}$")
    axs[1].plot(S0 * np.exp((mu - sg ** 2 / 2) * T), 0, "v", color=c, ms=6, clip_on=False)
axs[1].axvline(mean_th, color=GREY, ls="--", lw=0.9)
axs[1].annotate("mean, the same\nfor every $\\sigma$", (mean_th, 0.03), (170, 0.028), color=GREY, fontsize=7,
                arrowprops=dict(arrowstyle="->", color=GREY, lw=0.7))
axs[0].set(title="(a) same shocks, three volatilities", xlabel="$t$", ylabel="$S_t$", xlim=(0, T))
axs[1].set(title=r"(b) density of $S_1$ ($\blacktriangledown$ = median)", xlabel="$s$", ylim=(0, None))
axs[0].legend()
fig.tight_layout()
save(fig, "gbm_sigma")

# Strong error of Euler and Milstein against the exact GBM solution
Mf, kmax = 5000, 10
dWf = rng.standard_normal((Mf, 2 ** kmax)) * sqrt(T / 2 ** kmax)
exact = S0 * np.exp((mu - sig ** 2 / 2) * T + sig * dWf.sum(1))
hs, e_eu, e_mi = [], [], []
for k in range(2, kmax + 1):
    h = T / 2 ** k
    dWk = dWf.reshape(Mf, 2 ** k, -1).sum(2)
    se, sm = np.full(Mf, S0), np.full(Mf, S0)
    for i in range(2 ** k):
        d = dWk[:, i]
        se = se + mu * se * h + sig * se * d
        sm = sm + mu * sm * h + sig * sm * d + 0.5 * sig ** 2 * sm * (d * d - h)
    hs.append(h); e_eu.append(np.abs(se - exact).mean()); e_mi.append(np.abs(sm - exact).mean())
p_eu, p_mi = np.polyfit(np.log(hs), np.log(e_eu), 1)[0], np.polyfit(np.log(hs), np.log(e_mi), 1)[0]
assert 0.35 < p_eu < 0.65 and 0.8 < p_mi < 1.2, (p_eu, p_mi)
num["EulerOrder"], num["MilsteinOrder"] = f"{p_eu:.2f}", f"{p_mi:.2f}"

fig, ax = plt.subplots(figsize=(3.6, 2.6))
hs = np.array(hs)
ax.loglog(hs, e_eu, "o-", color=BLUE, ms=3.5, label=f"Euler--Maruyama (slope {p_eu:.2f})".replace("--", "-"))
ax.loglog(hs, e_mi, "s-", color=ORANGE, ms=3.5, label=f"Milstein (slope {p_mi:.2f})")
ax.loglog(hs, e_eu[-1] * (hs / hs[-1]) ** 0.5, color=GREY, ls=":", lw=0.9, label=r"$\propto\Delta t^{1/2}$")
ax.loglog(hs, e_mi[-1] * (hs / hs[-1]), color=GREY, ls="--", lw=0.9, label=r"$\propto\Delta t$")
ax.set(xlabel=r"step size $\Delta t$", ylabel=r"$\mathbb{E}|S_1^{\Delta t}-S_1|$")
ax.legend(fontsize=7)
save(fig, "gbm_error")

# ------------------------------------------------------------ 3. Ornstein-Uhlenbeck
th, m, s_ou, To, no = 2.0, 0.0, 1.0, 3.0, 600
dto = To / no
to = np.linspace(0, To, no + 1)
sd_inf = s_ou / sqrt(2 * th)
step = ou_step(th, m, s_ou, dto)
X = paths(1.5, step, rng.standard_normal((M, no)))
i1 = int(round(1 / dto))
check("OU", r"$\mathbb{E}[X_1]$", X[:, i1], m + (1.5 - m) * np.exp(-th))
check("OU", r"$\mathrm{Var}[X_1]$", (X[:, i1] - m - 1.5 * np.exp(-th)) ** 2, s_ou ** 2 / (2 * th) * (1 - np.exp(-2 * th)))
check("OU", r"$\mathrm{Var}[X_3]$ (near $\sigma^2/2\theta$)", (X[:, -1] - 1.5 * np.exp(-th * To)) ** 2,
      s_ou ** 2 / (2 * th) * (1 - np.exp(-2 * th * To)))
num["OUHalfLife"] = f"{np.log(2) / th:.2f}"

fig, axs = plt.subplots(1, 2, figsize=(6.3, 2.6))
for x0, c in zip([-1.5, 0.0, 1.5], [GREEN, GREY, BLUE]):
    Xs = X if x0 == 1.5 else paths(x0, step, rng.standard_normal((6, no)))
    axs[0].plot(to, Xs[:6].T, color=c, lw=0.55, alpha=0.8)
    mt = m + (x0 - m) * np.exp(-th * to)
    axs[0].plot(to, mt, color=c, lw=1.6, label=f"$X_0={x0:g}$")
sd_t = s_ou * np.sqrt((1 - np.exp(-2 * th * to)) / (2 * th))
axs[0].set(title="(a) three starting points", xlabel="$t$", ylabel="$X_t$", xlim=(0, To), ylim=(-2.1, 2.9))
axs[0].legend(loc="upper right", ncol=3, fontsize=7)
edges = np.linspace(-1.6, 2.4, 161)
H = np.stack([np.histogram(X[:, j], bins=edges, density=True)[0] for j in range(no + 1)], 1)
axs[1].imshow(H, origin="lower", aspect="auto", extent=(0, To, edges[0], edges[-1]), cmap="Blues", norm=PowerNorm(0.7, vmin=0, vmax=H[:, 30:].max()))
mt = m + (1.5 - m) * np.exp(-th * to)
axs[1].plot(to, mt, color=ORANGE, lw=1.3, label="mean")
```

```
axs[1].plot(to, mt + 2 * sd_t, color=ORANGE, ls="--", lw=0.9, label=r"mean␣$\pm␣2$␣sd")
axs[1].plot(to, mt - 2 * sd_t, color=ORANGE, ls="--", lw=0.9)
axs[1].set(title=r"(b)␣density␣of␣$X_t$␣from␣$X_0=1.5$", xlabel="$t$")
axs[1].grid(False)
axs[1].legend(loc="upper␣right")
fig.tight_layout()
save(fig, "ou_paths")

fig, axs = plt.subplots(1, 3, figsize=(6.3, 2.1))
x = np.linspace(-2, 2, 300)
axs[0].hist(X[:, -1], bins=70, density=True, color=BLUE, alpha=0.55, label="simulated")
axs[0].plot(x, npdf(x, m, sd_inf), color=DARK, lw=1.3, label=r"$N(\mu,\sigma^2/2\theta)$")
axs[0].set(title="(a)␣$X_3$␣vs␣stationary␣law", xlabel="$x$", ylim=(0, 1.15))
axs[0].legend(loc="upper␣left", fontsize=6.5)
lag = 400
Xs = paths(rng.normal(m, sd_inf, M), step, rng.standard_normal((M, lag))) # start in the stationary law
ac = [np.corrcoef(Xs[:, 0], Xs[:, j])[0, 1] for j in range(0, lag + 1, 10)]
tau = to[: lag + 1 : 10]
axs[1].plot(tau, ac, "o", color=BLUE, ms=3, label="simulated")
axs[1].plot(tau, np.exp(-th * tau), color=ORANGE, lw=1.2, label=r"$e^{-\theta\tau}$")
axs[1].set(title=r"(b)␣$\mathrm{corr}(X_0,X_\tau)$", xlabel=r"lag␣$\tau$")
axs[1].legend()
z1 = rng.standard_normal(no)[None]
for thv, c in zip([0.5, 2.0, 8.0], [GREEN, BLUE, RED]):
    axs[2].plot(to, paths(1.5, ou_step(thv, m, s_ou, dto), z1)[0], color=c, lw=0.8, label=rf"$\theta={thv:
        g}$")
axs[2].set(title=r"(c)␣same␣shocks,␣three␣$\theta$", xlabel="$t$", ylim=(None, axs[2].get_ylim()[1] + 1))
axs[2].legend(fontsize=6.5, ncol=3, loc="upper␣center", handlelength=1, columnspacing=0.6)
fig.tight_layout()
save(fig, "ou_stationary")

# ------------------------------------------------------------- 4. Short-rate models (VIR): Vasicek and
      CIR
a, b, sv, r0, Tr, nr = 0.5, 0.05, 0.02, 0.02, 10.0, 520 # weekly steps, 10 years
sc = sv / sqrt(b) # CIR vol chosen so both models share the same long-run variance
dtr = Tr / nr
tr = np.linspace(0, Tr, nr + 1)
num.update(CIRSigma=f"{sc:.4f}", FellerLHS=f"{2␣*␣a␣*␣b:.3f}", FellerRHS=f"{sc␣**␣2:.4f}")
assert 2 * a * b >= sc ** 2 # Feller condition: CIR never hits 0

Rv = paths(r0, ou_step(a, b, sv, dtr), rng.standard_normal((M, nr)))
e, c = np.exp(-a * dtr), sc ** 2 * (1 - np.exp(-a * dtr)) / (4 * a)
nu = 4 * a * b / sc ** 2
Rc = np.empty_like(Rv)
Rc[:, 0] = r0
for i in range(nr): # exact CIR step: scaled noncentral chi-square
    Rc[:, i + 1] = c * rng.noncentral_chisquare(nu, Rc[:, i] * e / c)

vmean = lambda tt, x0=r0: b + (x0 - b) * np.exp(-a * tt)
vvar = lambda tt: sv ** 2 / (2 * a) * (1 - np.exp(-2 * a * tt))
cvar = lambda tt: r0 * sc ** 2 / a * (np.exp(-a * tt) - np.exp(-2 * a * tt)) + b * sc ** 2 / (2 * a) * (1
     - np.exp(-a * tt)) ** 2


def bond_vasicek(tau, x0):
    B = (1 - np.exp(-a * tau)) / a
    return np.exp((b - sv ** 2 / (2 * a ** 2)) * (B - tau) - sv ** 2 * B ** 2 / (4 * a) - B * x0)


def bond_cir(tau, x0):
    h = sqrt(a * a + 2 * sc ** 2)
    den = (h + a) * (np.exp(h * tau) - 1) + 2 * h
    A = (2 * h * np.exp((a + h) * tau / 2) / den) ** (2 * a * b / sc ** 2)
    return A * np.exp(-2 * (np.exp(h * tau) - 1) / den * x0)


def integral(R): # trapezoid rule for int_0^t r ds
    return np.concatenate([np.zeros((M, 1)), np.cumsum((R[:, 1:] + R[:, :-1]) / 2 * dtr, 1)], 1)


Iv, Ic = integral(Rv), integral(Rc)
j1, j5 = int(1 / dtr), int(5 / dtr)
check("Vasicek", r"$\mathbb{E}[r_1]$", Rv[:, j1], vmean(1))
```

```
check("Vasicek", r"$\mathrm{Var}[r_1]$", (Rv[:, j1] - vmean(1)) ** 2, vvar(1))
check("Vasicek", r"$P(0,5)$", np.exp(-Iv[:, j5]), bond_vasicek(5, r0))
check("Vasicek", r"$P(0,10)$", np.exp(-Iv[:, -1]), bond_vasicek(10, r0))
pneg = float(Phi(-vmean(Tr) / sqrt(vvar(Tr))))
check("Vasicek", r"$\mathbb{P}(r_{10}<0)$", (Rv[:, -1] < 0).astype(float), pneg)
check("CIR", r"$\mathbb{E}[r_1]$", Rc[:, j1], vmean(1))
check("CIR", r"$\mathrm{Var}[r_1]$", (Rc[:, j1] - vmean(1)) ** 2, cvar(1))
check("CIR", r"$P(0,5)$", np.exp(-Ic[:, j5]), bond_cir(5, r0))
check("CIR", r"$P(0,10)$", np.exp(-Ic[:, -1]), bond_cir(10, r0))
assert Rc.min() > 0
num.update(PNegTh=f"{100␣*␣pneg:.2f}", PNegSim=f"{100␣*␣(Rv[:,␣-1]␣<␣0).mean():.2f}",
           PNegEver=f"{100␣*␣(Rv.min(1)␣<␣0).mean():.1f}", CIRMin=f"{100␣*␣Rc.min():.3f}",
           VasMin=f"{100␣*␣Rv.min():.2f}", YInf=f"{100␣*␣(b␣-␣sv␣**␣2␣/␣(2␣*␣a␣**␣2)):.2f}")

fig, axs = plt.subplots(1, 2, figsize=(6.3, 2.6), sharey=True)
for ax, R, name in [(axs[0], Rv, "(a)␣Vasicek"), (axs[1], Rc, "(b)␣CIR")]:
    fan(ax, tr, 100 * R, BLUE)
    ax.plot(tr, 100 * vmean(tr), color=ORANGE, ls="--", lw=1.2, label="theoretical␣mean")
    ax.axhline(0, color=RED, lw=0.8)
    ax.set(title=name, xlabel="$t$␣(years)", xlim=(0, Tr))
axs[0].set_ylabel("short␣rate␣$r_t$␣(%)")
fig.tight_layout()
fig.legend(*axs[0].get_legend_handles_labels(), ncol=4, loc="lower␣center", bbox_to_anchor=(0.5, 1.0))
save(fig, "rate_paths")

fig, axs = plt.subplots(1, 3, figsize=(6.3, 2.2))
x = np.linspace(-0.03, 0.14, 400)
axs[0].hist(100 * Rv[:, -1], bins=80, density=True, color=BLUE, alpha=0.45, label="Vasicek")
axs[0].hist(100 * Rc[:, -1], bins=80, density=True, color=ORANGE, alpha=0.45, label="CIR")
axs[0].plot(100 * x, npdf(x, vmean(Tr), sqrt(vvar(Tr))) / 100, color=DARK, lw=1.1)
axs[0].axvspan(-3, 0, color=RED, alpha=0.08)
axs[0].set(title="(a)␣$r_{10}$␣(%)", xlim=(-3, 14))
axs[0].legend(fontsize=7)
tt = tr[1:]
axs[1].plot(tt, 100 * Phi(-vmean(tt) / np.sqrt(vvar(tt))), color=DARK, lw=1.3, label="theory")
axs[1].plot(tt[::13], 100 * (Rv[:, 1::13] < 0).mean(0), "o", color=BLUE, ms=2.5, label="Vasicek␣sim.")
axs[1].plot(tt[::13], 100 * (Rc[:, 1::13] < 0).mean(0), "s", color=ORANGE, ms=2.5, label="CIR␣sim.")
axs[1].set(title=r"(b)␣$\mathbb{P}(r_t<0)$␣(%)", xlabel="$t$")
axs[1].legend(fontsize=6.5)
taus = np.linspace(0.05, 10, 200)
for x0, ls in [(0.02, "-"), (0.05, "--"), (0.09, ":")]:
    axs[2].plot(taus, -100 * np.log(bond_vasicek(taus, x0)) / taus, color=BLUE, ls=ls, lw=1.1)
    axs[2].plot(taus, -100 * np.log(bond_cir(taus, x0)) / taus, color=ORANGE, ls=ls, lw=1.1)
yrs = np.arange(1, 11)
axs[2].plot(yrs, -100 * np.log(np.exp(-Iv[:, yrs * 52]).mean(0)) / yrs, "o", color=BLUE, ms=3)
axs[2].plot(yrs, -100 * np.log(np.exp(-Ic[:, yrs * 52]).mean(0)) / yrs, "s", color=ORANGE, ms=3)
axs[2].set(title="(c)␣yield␣curve␣$y(0,\\tau)$␣(%)", xlabel=r"maturity␣$\tau$")
axs[2].text(3.2, 1.0, "$r_0=2\\%$␣solid,␣$5\\%$␣dashed,\n$9\\%$␣dotted;␣marks:␣Monte␣Carlo", fontsize=6.3,
      color=GREY)
axs[2].set_ylim(0.8, None)
fig.tight_layout()
save(fig, "rate_compare")

# ------------------------------------------------------------------ overview: one path of each process
fig, axs = plt.subplots(1, 4, figsize=(6.3, 1.7))
for ax, (y, tx, ttl) in zip(axs, [(W[0], t, "Brownian␣motion"), (S[0], t, "geometric␣BM"),
                                  (X[0], to, "Ornstein-Uhlenbeck"), (100 * Rv[0], tr, "Vasicek␣rate␣(%)")]):
    ax.plot(tx, y, color=BLUE, lw=0.7)
    ax.set_title(ttl, fontsize=8)
    ax.tick_params(labelsize=7)
fig.tight_layout()
save(fig, "overview")

# ------------------------------------------------------------------ tables
with open(os.path.join(TAB, "moments.tex"), "w") as f:
    last = None
    for g, name, th_, est, se, z in rows:
        if last and g != last:
            f.write("\\midrule\n")
        sci = lambda v, p: "$" + f"{v:.{p}g}".replace("e-0", r"\times10^{-").replace("e-", r"\times10^{-")
            + ("}" if "e" in f"{v:.{p}g}" else "") + "$"
        f.write(f"{g␣if␣g␣!=␣last␣else␣''}␣&␣{name}␣&␣{sci(th_,␣5)}␣&␣{sci(est,␣5)}␣&␣{sci(se,␣2)}␣&␣${z
            :+.2f}$␣\\\\\n")
```

```
        last = g
    f.write("\\bottomrule\n") # inside the file: \input followed by \bottomrule breaks \noalign
num["MaxZ"] = f"{max(abs(r[-1]) for r in rows):.2f}"
num["NChecks"] = str(len(rows))
with open(os.path.join(TAB, "numbers.tex"), "w") as f:
    for k, v in num.items():
        f.write(f"\\newcommand{{\\{k}}}{{{v}}}\n")
print(f"{len(rows)} checks passed, max |z| = {num['MaxZ']}")
print(num)
```